Research Notes: Origin Identification of LLM-Generated Text

Working notes (mostly written by Claude Code)

October 19, 2026

Contents

1 Regulatory background **1**
- 1.1 EU AI Act, Article 50 (transparency obligations) 1
- 1.2 Why this matters for the thesis 1
- 1.3 Open items 1

2 Text watermarking: algorithms **2**

3 Statistics-based detection: count-based n-gram methods **10**

4 Adversarial attacks on detectors **11**

5 Article summaries **17**

6 Ideas and future research **17**

1 Regulatory background

1.1 EU AI Act, Article 50 (transparency obligations)

Source: text of Art. 50 as pasted by the author (the paste carried an "AMENDED" marker on para. 7). **TODO:** verify against the Official Journal version and any later amendments.

Art. 50(2) — core obligation for this thesis. Providers of AI systems (including general-purpose ones) generating synthetic audio, image, video *or text* must ensure outputs are

- **marked in a machine-readable format**, and
- **detectable** as artificially generated or manipulated.

Technical solutions must be *effective, interoperable, robust and reliable* "as far as technically feasible", taking into account content-type specificities, implementation costs and the generally acknowledged state of the art (possibly reflected in technical standards).

Exemptions: the AI system performs an assistive function for standard editing; does not substantially alter the deployer's input or its semantics; or use is authorised by law for criminal-offence detection/prosecution.

Art. 50(4), second subparagraph — deployer duty for text. Deployers of AI that generates/manipulates text published to inform the public on matters of public interest must disclose its artificial origin. Not applicable if the content underwent human review or editorial control and a person holds editorial responsibility (or for law-enforcement uses).

Art. 50(5). Information must be clear and distinguishable, at the latest at first interaction/exposure, and meet accessibility requirements.

Art. 50(7) — codes of practice. The Commission encourages codes of practice on detection, marking and labelling of AI-generated content, assesses their adequacy (Art. 56(6)), and may adopt an implementing act with common rules (Art. 98(2)) if a code is inadequate.

1.2 Why this matters for the thesis

- The law demands *robustness* and *reliability* "as far as technically feasible" — directly motivates the adversarial evaluation: a watermark that dies under paraphrase arguably fails the "robust" requirement.
- "Machine-readable marking" for plain text cannot rely on metadata (strippable by copy-paste), so the signal must live *in the text*: watermarking in generation, plus fingerprinting as a fallback for unmarked/unknown-source text.
- The duty falls on *providers*; detection by third parties must work with limited access to the generator, which motivates black-box fingerprint features and probabilistic fusion.
- The "assistive editing / no substantial change" exemption makes the human+AI mixed and lightly edited text cases legally and technically relevant.

1.3 Open items

- Current status and content of the Code of Practice on marking/labelling AI-generated content.
- Application dates for Art. 50 and any postponement.
- C2PA and why metadata-based provenance does not cover plain text.

2 Text watermarking: algorithms

Sources: original papers in `articles/pdfs/` (kgw, unigram, sweet, ewd, sir, upv, exp_edit, semstamp, k_semstamp); taxonomy from Wu et al. (CL 2025). Notation: V vocabulary, ℓ_t logits, $p_t = \mathrm{softmax}(\ell_t)$, y_t emitted token, T number of scored tokens, γ green fraction, δ bias, $|s|_G = \sum_t \mathbf{1}[y_t \in G_t]$. The common detection statistic is

$$z = \frac{|s|_G - \gamma T}{\sqrt{T\gamma(1-\gamma)}}. \tag{1}$$

Algorithm 1 KGW soft red list (Kirchenbauer et al. 2023)

Require: prompt, green fraction $\gamma \in (0,1)$, strength $\delta > 0$, threshold τ $(= 4)$

procedure GENERATE
  for $t = 0, 1, \ldots$ **do**
    $\ell_t \leftarrow M(\text{prompt}, y_{<t})$
    seed RNG with hash(y_{t-1})
    partition V into G_t ($|G_t| = \gamma|V|$) and R_t ($(1-\gamma)|V|$)
    $\hat{\ell}_{t,v} \leftarrow \ell_{t,v} + \delta\,\mathbf{1}[v \in G_t]$ for all v
    $y_t \sim \mathrm{softmax}(\hat{\ell}_t)$
  end for
end procedure
procedure DETECT$(y_{1:T})$
  $|s|_G \leftarrow \sum_t \mathbf{1}[y_t \in G_t]$, G_t recomputed from hash(y_{t-1})
  $z \leftarrow$ Eq. (1)
  return $z > \tau$
end procedure

\+ no model/API access needed at detection; low false-positive rate; degrades gracefully under attack; detected after about 800 tokens even with attacks (Kirchenbauer et al. 2023b)

− distorts the output distribution (δ); weak on low-entropy text; with context h one edited token changes h lists (attack amplification, or small h is insecure); paraphrasing weakens it

Algorithm 2 KGW self-hash: choose token y_t (window h, key K, boost δ)

sort V by logit, highest first: $v_0, v_1, \ldots$ with logits $\ell_0 \geq \ell_1 \geq \ldots$ # v_k = one token, ℓ_k = its logit
$k \leftarrow 0$
while true **do**
  $y_t \leftarrow v_k$ # candidate
  $H_i \leftarrow F_K(y_t, y_{t-i})$ for $i = 1..h$ # hash of candidate + one earlier token
  $seed \leftarrow \min_i H_i$
  $bit \leftarrow \mathrm{PRG}(seed)$ # 1 = green (prob. γ), 0 = red
  if $bit = 1$ **then**
    return y_t # highest-logit green token
  else if $\ell_{k+1} < \ell_0 - \delta$ **then**
    return v_0 # no green token within δ of the top: take the top (red) token
  else
    $k \leftarrow k + 1$ # next candidate is within δ of the top: try it
  end if
end while

\+ wide window h without attack amplification (a changed token alters position t w.p. only $1/h$); $|V|^2$ distinct lists hide the rule

− greedy per-token search over sorted logits (extra cost); still a quality-reducing logit bias; falls back to red token if no green token within δ

Algorithm 3 Unigram-Watermark (Zhao et al. 2023)

Require: RNG F, γ, δ, threshold τ $(= 6)$

procedure GENERATE
  key $\mathsf{k} \leftarrow F$; fix $G \subset V$, $|G| = \gamma|V|$, $R = G^c$ # once
  $\hat{\ell}_{t,v} \leftarrow \ell_{t,v} + \delta\,\mathbf{1}[v \in G]$ for every prefix
end procedure
procedure DETECT$(y_{1:n}, \mathsf{k})$
  $|y|_G \leftarrow \sum_{t=1}^{n} \mathbf{1}[y_t \in G]$
  $z_y \leftarrow (|y|_G - \gamma n)/\sqrt{n\gamma(1-\gamma)}$
  return $z_y > \tau$
end procedure

\+ provable quality, type-I/II error and security guarantees; linear time; no model or prompt at detection; robust to cropping and minor edits

− single fixed green list: readily reverse-engineered by about $|V|$ detection-API queries; same token always same colour; larger δ lowers quality

Algorithm 4 SWEET (Lee et al. 2023)

Require: entropy threshold $\tau_H \in [0, \log|V|]$, γ, δ, z_{thr}

procedure GENERATE
  for $t = 0, 1, \ldots$ **do**
    $p_t \leftarrow \text{softmax}(\ell_t)$; $H_t \leftarrow -\sum_v p_{t,v} \log p_{t,v}$
    if $H_t > \tau_H$ **then**
      seed RNG with $\text{hash}(y_{t-1})$; split $V \to G_t, R_t$
      $\ell_{t,v} \leftarrow \ell_{t,v} + \delta$ for $v \in G_t$
    end if
    $y_t \sim \text{softmax}(\ell_t)$
  end for
end procedure
procedure DETECT$(x, y_{0:N-1})$
  $N^h \leftarrow 0$, $N^h_G \leftarrow 0$
  for $t = 0, \ldots, N-1$ **do**
    recompute H_t # needs model and prompt
    if $H_t > \tau_H$ **then**
      $N^h \leftarrow N^h + 1$; recover G_t from $\text{hash}(y_{t-1})$
      if $y_t \in G_t$ **then**
        $N^h_G \leftarrow N^h_G + 1$
      end if
    end if
  end for
  $z \leftarrow \dfrac{N^h_G - \gamma N^h}{\sqrt{N^h \gamma(1-\gamma)}}$; **return** $z > z_{\text{thr}}$
end procedure

\+ keeps code correct by skipping low-entropy tokens; >10% AUROC over WLLM and EXP-EDIT; stays detectable on short code

− detection needs the model and prompt to recompute entropy; entropy threshold τ_H must be tuned on human code; designed for code

Algorithm 5 EWD detection (Lu et al. 2024; generation = KGW)

Require: LM M, prompt x, text $T = (T_0, \ldots, T_{n-1})$, key k, context window m, threshold τ, green fraction γ, spike modulus ρ (called τ in EWD, z in KGW), offset C_0, increasing function f

for $l = m, \ldots, n-1$ **do**
  $p_l \leftarrow \text{softmax}\big(M(x, T_{:l-1})\big)$ # next-token distribution at step l, a vector over V
  $SE_l \leftarrow \sum_{v \in V} \dfrac{p_{l,v}}{1 + \rho\, p_{l,v}}$ # spike entropy: ≈ 1 if flat, $\approx 1/(1+\rho)$ if one token takes all
  $W_l \leftarrow f(SE_l - C_0)$ # weight of token l (paper analysis: f linear, $W_l = SE_l - C_0$)
end for
$S \leftarrow 0$ # weighted green count (called $|s|_G$ in the paper)
for $l = m, \ldots, n-1$ **do**
  $G \leftarrow$ green list from key k and $T_{l-m:l-1}$ # as in KGW
  if $T_l \in G$ **then**
    $S \leftarrow S + W_l$
  end if
end for
$z' \leftarrow \dfrac{S - \gamma \sum_{l=m}^{n-1} W_l}{\sqrt{\gamma(1-\gamma) \sum_{l=m}^{n-1} W_l^2}}$
return $z' > \tau$ # watermarked?

\+ no manual entropy threshold; every token weighted by its entropy; better on low-entropy text; robust to back-translation; same performance on high-entropy text

− detection-only (generation stays KGW, so inherits its weaknesses); needs model logits at detection; weight function f and C_0 are design choices

Algorithm 6 SIR (Liu et al. 2024)

Require: strength δ, embedder E (BERT), trained watermark model $\mathcal{T}$, scalars $k_1, k_2, R, \lambda_1, \lambda_2$

procedure GENERATE
  $P_M \leftarrow M(x^{\text{prompt}}, t_{:l-1})$; $e_l \leftarrow E(t_{:l-1})$
  $P_W \leftarrow \tanh(k_2 \mathcal{T}(e_l)) \in [-1, 1]^{|V|}$
  $P_{\hat{M}} \leftarrow P_M + \delta P_W$; sample next token
end procedure
procedure DETECT$(t_{1:N})$
  $z \leftarrow \frac{1}{N} \sum_{j=1}^{N} P_W^{(t_j)}(x^{\text{prompt}}, t_{:j-1})$ # null: mean 0, sd 1
  return $z >$ threshold
end procedure
procedure TRAIN $\mathcal{T}$
  $L_s \leftarrow \sum_{i,j} \Big| \cos(\mathcal{T}(e_i), \mathcal{T}(e_j)) - \tanh\big(k_1(\cos(e_i, e_j) - \overline{\cos(e, e')})\big) \Big|$
  $L_n \leftarrow \sum_i \big| \sum_j \mathcal{T}(e_i)_{(j)} \big| + \sum_i \big| \sum_j \mathcal{T}(e_j)_{(i)} \big| + \lambda_1 \sum_{i,j} \big| R - \mathcal{T}(e_j)_{(i)} \big|$
  minimise $L = L_s + \lambda_2 L_n$
end procedure

\+ robust to synonym substitution and paraphrase while keeping security; X-SIR extends to translation

− needs a trained watermark model and an embedder at both generation and detection; null hypothesis only approximately true, no guarantee out of domain

Algorithm 7 UPV (Liu et al. 2024)

Require: LM M, window w, top-K, boost δ, green fraction γ; two small trained networks:
E: token embedding, *not* BERT. A 5-layer MLP: 16-bit binary token ID $\to \mathbb{R}^{64}$
FFN: 3-layer MLP $\mathbb{R}^{64w} \to 64 \to 64 \to 1$, then sigmoid and rounding to $\{0,1\}$
LSTM: 5-layer, input 64, hidden 128, linear output $\to 1$, sigmoid (detector)
$a \oplus b$ = vector concatenation

procedure GENERATE($x_{0:n-1}$) # choose next token x_n
  $\ell \leftarrow M(x_{0:n-1})$; $C \leftarrow$ the K tokens with the highest logit
  for each candidate $c \in C$ **do**
    $r_c \leftarrow \mathrm{FFN}\big(E(x_{n-w+1}) \oplus \cdots \oplus E(x_{n-1}) \oplus E(c)\big)$ # 1 = green, 0 = red
  end for
  $\ell_c \leftarrow \ell_c + \delta \cdot r_c$ for $c \in C$; sample x_n from softmax(ℓ) # top-K or beam search
end procedure
procedure DETECTWITHGENERATOR($x_{0:n-1}$) # needs the generation net W=(E,FFN)
  $r_t \leftarrow \mathrm{FFN}\big(E(x_{t-w+1}) \oplus \cdots \oplus E(x_t)\big)$ for $t = w-1, \ldots, n-1$
  $T \leftarrow n - w + 1$ # number of scored windows; $|s|_G \leftarrow \sum_t r_t$
  $z \leftarrow \dfrac{|s|_G - \gamma T}{\sqrt{\gamma(1-\gamma)T + \sigma^2 T}}$ # σ: measured sd of the green ratio of W, since W gives only $\approx \gamma$
end procedure
procedure DETECTPUBLIC($x_{0:n-1}$) # public: only the detector net, no W and no key
  return $\mathrm{LSTM}\big(E(x_0) \oplus \cdots \oplus E(x_{n-1})\big) > 0.5$ # E: copied from the generator, frozen
end procedure

training: W is trained on windows with a fraction γ of positive labels; the LSTM is trained on random token-ID sequences, labeled 1 iff the z-score from W exceeds a threshold (sequence treated as cyclic)

+ public verification without the generation key; lower FPR and better paraphrase resistance than WLLM

− detector network slightly worse than key-based z-test (e.g. FNR up to 4.7% on OPT-1.3B); two networks to train; green ratio $\hat{\gamma}$ only approximately constant; spoofing via token frequencies only hard for large windows

Algorithm 8 EXP-EDIT family (Kuditipudi et al. 2023)

Require: model M, variant $\in$ {ITS, EXP}, secret key $\xi = (\xi_1, \ldots, \xi_n)$ (shared with detector, used cyclically, e.g. $n = 256$); output length m, block size k, edit penalty γ, resamples T_r, key law ν (how keys are drawn)
ITS: $\xi_j = (u_j, \pi_j)$, $u_j \sim \mathrm{Unif}[0,1]$, π_j random permutation of V ($\pi_j(v)$ = position of v)
EXP: $\xi_j \in [0,1]^{|V|}$, one uniform number $\xi_{j,v}$ per token v

procedure GENERATE(prompt)
  $\tau \sim \mathrm{Unif}\{0, \ldots, n-1\}$ # random start in the key, so different texts use different parts
  for $i = 1, \ldots, m$ **do**
    $\mu \leftarrow \mathrm{softmax}\big(M(\mathrm{prompt}, y_{<i})\big)$; $j \leftarrow (\tau + i) \% n$
    if ITS **then**
      $order[\pi_j[v]] \leftarrow v$ for every token ID $v \in \{0..|V|-1\}$ # $order[r]$ = ID of the token with rank r
      $s \leftarrow 0$
      for $r = 0, 1, \ldots, |V|-1$ **do**
        $v \leftarrow order[r]$; $s \leftarrow s + \mu[v]$ # $\mu[v]$ = probability of token v
        if $s \geq u_j$ **then**
          $y_i \leftarrow v$; **break** # y_i is one token ID (an integer)
        end if
      end for
    else
      $y_i \leftarrow \arg\min_{v \in V} \; -\log(\xi_{j,v}) / \mu(v)$
    end if
  end for
  return y
end procedure
procedure DETECT(y) # y of any length, possibly edited or cropped; returns a p-value
  $\mathrm{cost}(v, j) \leftarrow |u_j - \frac{\pi_j(v)-1}{|V|-1}|$ (ITS) or $-\log \xi_{j,v}$ (EXP) # how badly token v fits key position j
  for each key $\xi' \in \{\xi, \xi^{(1)}, \ldots, \xi^{(T_r)}\}$ with $\xi^{(t)} \sim \nu$ fresh random keys **do**
    $\phi(\xi') \leftarrow \infty$
    for each block start $i \in \{1..|y|-k+1\}$ and key start $j \in \{1..n\}$ **do**
      $D[a][0] \leftarrow \gamma a$; $D[0][b] \leftarrow \gamma b$ for $0 \leq a, b \leq k$
      $D[a][b] \leftarrow \min\big(D[a-1][b-1] + \mathrm{cost}(y_{i+a-1}, (j+b-1)\%n),\ D[a-1][b] + \gamma,\ D[a][b-1] + \gamma\big)$
      $\phi(\xi') \leftarrow \min\big(\phi(\xi'), D[k][k]\big)$ # best-matching block (edit distance)
    end for
  end for
  return $\hat{p} = \dfrac{1 + |\{t : \phi(\xi^{(t)}) \leq \phi(\xi)\}|}{T_r + 1}$ # small $\hat{p}$ = text fits the true key much better than random keys
end procedure

+ distortion-free; exact non-asymptotic p-value; robust to editing and cropping (about 40–50% of tokens changed)

− detection cost $O(mnk^2)$ plus permutation resampling; same text for same key unless shifted; key reuse across queries re-introduces distortion unless $n = \omega(mT^2)$

Algorithm 9 SemStamp (Hou et al. 2023)

Require: LM P_{LM}, prompt $s^{(0)}$, T sentences; encoder M_{embd} (dim h), LSH dim d, γ, margin m, $N_{\max}$, prime p

function SIG(s)
  $v \leftarrow M_{\text{embd}}(s)$; **return** $[\mathbf{1}[n^{(i)} \cdot v > 0]]_{i=1}^{d}$
end function
function MARGIN(s, m)
  $v \leftarrow M_{\text{embd}}(s)$; **return** $\min_i |\cos(v, n^{(i)})| \geq m$
end function
procedure GENERATE
  draw $n^{(1)}, \ldots, n^{(d)} \in \mathbb{R}^h$ # 2^d regions
  for $t = 1, \ldots, T$ **do**
    seed $\leftarrow [\text{SIG}(s^{(t-1)})]_{10} \cdot p$
    split $\{0,1\}^d$ into valid $G^{(t)}$ ($\gamma 2^d$) and blocked $R^{(t)}$
    repeat
      sample $s^{(t)} \sim P_{\text{LM}}(\cdot \mid s^{(<t)})$
    until $(\text{SIG}(s^{(t)}) \in G^{(t)}$ **and** $\text{MARGIN}(s^{(t)}, m))$ **or** $N_{\max}$ tries
  end for
end procedure
procedure DETECT(text)
  split into $s_1..s_N$; $S_V \leftarrow \sum_{t \geq 2} \mathbf{1}[\text{SIG}(s_t) \in G^{(t)}(s_{t-1})]$
  return $z = \dfrac{S_V - \gamma N}{\sqrt{\gamma(1-\gamma)N}}$
end procedure

\+ robust to paraphrase, including the bigram paraphrase attack; better quality than token-level WLLM
− rejection sampling: about 13.8 candidate sentences per valid one; fails on sentence reordering (DIPPER) and when machine text is a small part of human text; needs a fine-tuned encoder

Algorithm 10 k-SemStamp generation and detection (Hou et al. 2024)

Require: LM, prompt $s^{(0)}$, domain $\mathcal{D}$, T, $N_{\max}$, margin m, γ, K clusters, prime p, N samples

function INITIALIZE($\mathcal{D}, K$)
  $\mathcal{D}'_N \sim \mathcal{D}$; **return** $C_K \leftarrow k\text{-means}(\mathcal{D}'_N, K)$
end function
function ASSIGN(s, C_K)
  return $\arg\min_{i \leq K} d_{\cos}(v, c_i)$, $v = M^{\mathcal{D}}_{\text{embd}}(s)$
end function
procedure GENERATE
  $C_K \leftarrow \text{INITIALIZE}(\mathcal{D}, K)$
  for $t = 1, \ldots, T$ **do**
    $q^{(t-1)} \leftarrow \text{ASSIGN}(s^{(t-1)}, C_K)$; seed $\leftarrow q^{(t-1)} \cdot p$
    split cluster indices into valid $G^{(t)}$ (γK) and blocked $R^{(t)}$ ($(1-\gamma)K$)
    repeat
      sample $s^{(t)}$ from LM
    until $q^{(t)} \in G^{(t)}$ **and** $d_{\cos}(v, c_q) < \min_{i \neq q} d_{\cos}(v, c_i) - m$, **or** $N_{\max}$ tries
  end for
end procedure
procedure DETECT(T, C_K)
  $s_1..s_N \leftarrow$ sentence-tokenise T; $q^{(1)} \leftarrow \text{ASSIGN}(s_1, C_K)$; $S_V \leftarrow 0$
  $G^{(1)} \leftarrow \text{RANDOMSAMPLE}(q^{(1)}p, K, \gamma)$
  for $t = 2, \ldots, N$ **do**
    $q^{(t)} \leftarrow \text{ASSIGN}(s_t, C_K)$; **if** $q^{(t)} \in G^{(t-1)}$ **then** $S_V \leftarrow S_V + 1$
    $G^{(t)} \leftarrow \text{RANDOMSAMPLE}(q^{(t)}p, K, \gamma)$
  end for
  return $z = \dfrac{S_V - \gamma N}{\sqrt{\gamma(1-\gamma)N}}$
end procedure

\+ robust to paraphrase, including the bigram paraphrase attack; better quality than token-level WLLM
− rejection sampling: about 13.8 candidate sentences per valid one; fails on sentence reordering (DIPPER) and when machine text is a small part of human text; needs a fine-tuned encoder

Data-driven (backdoor) watermarks

Algorithm 11 Backdoor watermark for pre-trained LMs (Gu et al. 2022)

Require: PLM $f(\cdot;\theta)$, proxy dataset D^{pro} per task $i = 1..K$, trigger (rare word, e.g. "cf", or common-word combination, e.g. "green idea nose"), target label y_i^T, trigger-embedding params θ_{E_w}, epochs T, rate α, task weights λ_i $(= 1)$, threshold

procedure EMBED
  $D_i^w \leftarrow \{(W(x), y_i^T) \mid (x,y) \in D_i^{\text{pro}}\}$, W inserts the trigger into a random portion of samples
  # combination trigger: also add $(W^*(x), y)$, W^* inserts a random sub-sequence, with the true label
  $\theta_i^{\text{clean}} \leftarrow \arg\min_\theta \sum_{(x,y)\in D_i^{\text{pro}}} L(f_i(x;\theta), y)$
  $ori_norm \leftarrow \|\theta_{E_w}\|_2$
  for $t = 1..T$ **do**
    for $i = 1..K$ **do**
      sample batch x^* from D_i^w; $L_i \leftarrow L(f_i(x^*; \theta_{E_w}, \theta_r), y_i^T)$; $g_i \leftarrow \nabla_{\theta_{E_w}} L_i$
    end for
    $g \leftarrow \sum_i \lambda_i g_i$; $\theta_{E_w} \leftarrow \theta_{E_w} - \alpha g$; $\theta_{E_w} \leftarrow \theta_{E_w} \cdot ori_norm / \|\theta_{E_w}\|_2$
  end for
  release $f(\cdot;\theta^*)$ with $\theta^* = (\theta \setminus \theta_{E_w}) \cup \theta^*_{E_w}$ # only trigger embedding changed
end procedure
procedure VERIFY(target model $f(\cdot;\theta^T)$, D^w)
  WESR $\leftarrow \left|\{x \in D^w : f(x;\theta^T) = y^T\}\right| / |D^w|$; **return** WESR $>$ threshold # black-box queries
end procedure

+ ownership checkable in a black-box setting; survives later fine-tuning (WESR $\approx$ 100% on IMDB, SST-2, MNLI, SNLI, PAWS with little accuracy loss); multi-task variant; common-word trigger is harder to filter than a rare word

− marks the *model*, not individual generated texts; rare-word triggers are detectable and filterable; trigger can be recovered with weight access (Lucas and Havens 2023); needs proxy datasets and retraining

Algorithm 12 Clean-label backdoor watermark for datasets (Tang et al. 2023)

Require: dataset $D_{ori} = \{(x_i, y_i)\}$, target class C, pre-trained model f, steps T, trigger t

function TEXTPERTURB($x = [v_1..v_m], y, f$)
  $A \leftarrow \emptyset$
  for $1 \le i \le |x|$ **do**
    $a \leftarrow$ highest-scoring of $\{\text{replace}(x,i), \text{insert}(x,i)\}$; $A \leftarrow A \cup \{a\}$
  end for # replace: WordNet synonym; insert: extra word; keep sentence-embedding cosine close
  for $1 \le t \le T$ **do**
    $a \leftarrow$ highest-scoring in A; $A \leftarrow A \setminus \{a\}$; $x^{(t)} \leftarrow \text{apply}(a, x^{(t-1)})$; **if** $f(x^{(t)}) \ne y$ **then return** $x^{(t)}$
  end for
  return $x^{(T)}$
end function
procedure MARK
  $D_{wm} \subset D_{ori}$, a small portion of class C only (label unchanged)
  $\hat{x} \leftarrow$ TEXTPERTURB(x) for $x \in D_{wm}$ # disables natural features; images/audio: PGD $x^{(t+1)} = \Pi_{clip}(x^{(t)} + \alpha\,\text{sgn}\nabla_x L)$, $\|\hat{x} - x\|_\infty \le \epsilon$
  stamp trigger: word-level (insert dictionary word at start/middle/end) or style-level (e.g. casual $\rightarrow$ formal)
end procedure
procedure VERIFY(suspect f, D_{test}, trigger t)
  $p = f_C(x_t)$, $q = f_C(x)$ # target-class softmax with / without trigger
  Wilcoxon signed-rank test of $H_0 : p - q < \alpha$; reject at level 0.05 $\Rightarrow$ dataset was used
end procedure

+ label-consistent samples, so not visibly wrong; about 1% watermarked samples suffice; works for text, image and audio; non-i.i.d.-safe Wilcoxon test

− protects a *dataset* (detects training on it), not LLM output; classification setting; word triggers are visible; needs a surrogate model for the perturbation and model retraining

Post-processing watermarks

Algorithm 13 Black-box synonym watermark (Yang et al. 2023)

Require: text $\mathbf{T} = (w_1..w_n)$, hash h (SHA-256), BERT, thresholds $\tau_{\text{sent}}, \tau_{\text{word}}$, weight λ, z-level Z_α, K candidates

function BIT(w_i, w_{i-1})
  return RandomBinary($h(w_i) \oplus h(w_{i-1})$)
end function
function CANDIDATES($\mathbf{T}, w_i$)
  $\tilde{w}_i \leftarrow$ dropout on the embedding of w_i; $C \leftarrow$ top-K BERT predictions for $\tilde{w}_i$ in context
  $C' \leftarrow \emptyset$
  for $s \in C$ **do**
    $\mathbf{T}' \leftarrow \mathbf{T}$ with $w_i \to s$
    $S_{\text{sent}} \leftarrow \cos(\text{SentEmb}(\mathbf{T}), \text{SentEmb}(\mathbf{T}'))$ # RoBERTa-MNLI
    $S_{\text{global}} \leftarrow \cos(\text{w2v}(w_i), \text{w2v}(s))$; $S_{\text{context}} \leftarrow \frac{1}{L}\sum_{l=1}^{L} \cos(f^l(w_i, c_{\mathbf{T}}), f^l(s, c_{\mathbf{T}'}))$
    $S_{\text{word}} \leftarrow \lambda S_{\text{context}} + (1-\lambda) S_{\text{global}}$
    if $S_{\text{sent}} \geq \tau_{\text{sent}}$ **and** $S_{\text{word}} \geq \tau_{\text{word}}$ **then**
      $C' \leftarrow C' \cup \{s\}$
    end if
  end for
  return C'
end function
procedure INJECT($\mathbf{T}$)
  for $i = 2, \ldots, n$ **do**
    if POSFILTER(w_i) **and** BIT(w_i, w_{i-1}) $= 0$ **then**
      $C' \leftarrow$ CANDIDATES($\mathbf{T}, w_i$)
      $w_i \leftarrow \arg\max_{s'_k \in C',\ b_k = 1} S_{\text{word}}(s'_k, w_i)$, $b_k = \text{RandomBinary}(h(s'_k) \oplus h(w_{i-1}))$
    end if
  end for
end procedure
procedure DETECT($\mathbf{T}$)
  $N \leftarrow 0$, $c \leftarrow 0$
  for $i = 2, \ldots, n$ **do**
    if POSFILTER(w_i) **and** (fast **or** CANDIDATES($\mathbf{T}, w_i$) $\neq \emptyset$) **then** # precise mode
      $N \leftarrow N + 1$; **if** BIT(w_i, w_{i-1}) $= 1$ **then** $c \leftarrow c + 1$
    end if
  end for
  $\hat{p} \leftarrow c/N$; $Z \leftarrow \dfrac{\hat{p} - p_0}{\sqrt{p_0(1-p_0)/N}}$, $p_0 = 0.5$; **return** $Z > Z_\alpha$
end procedure

\+ works on black-box LLMs, third party can add it; robust to retranslation, polishing, word deletion, synonym substitution; meaning preserved

− relies on POS exclusion lists and several large models (BERT, RoBERTa, GloVe); precise detection costs candidate generation per word; synonym rules are a systematic pattern an attacker can target

Algorithm 14 Context-aware lexical substitution watermark (Yang et al. 2022)

Require: sentence $S = \{t_1..t_N\}$, BERT$_{gen}$, BERT$_{score}$ (RoBERTa-MNLI), $K = 32$, spacing $f = 1$, bits m

function LS(S, i) # ranked candidates RW for t_i
  $S_{\setminus i} \leftarrow \{t_1..t_{i-1}, [\text{MASK}], t_{i+1}..t_N\}$; $I_i \leftarrow S\,[\text{SEP}]\,S_{\setminus i}$
  $W \leftarrow$ top-K words from BERT$_{gen}(I_i)$ (morphological derivatives of t_i removed)
  for $w \in W$ **do**
    $\hat{S} \leftarrow S$ with $t_i \to w$; $I'_i \leftarrow S\,[\text{SEP}]\,\hat{S}$; $SR_w \leftarrow$ BERT$_{score}(I'_i)$ # P(entailment)
  end for
  return W sorted by SR_w descending
end function
function ST(i, S) # synchronicity test, Eq. (6)
  $FC = \{w'_1, w'_2\} \leftarrow$ top-2 of LS(S, i) with $SR > 0.95$
  $\hat{S}_j \leftarrow S$ with $t_i \to w'_j$; $FC_j \leftarrow$ same procedure on $\hat{S}_j$, $j = 1, 2$
  $Sync \leftarrow [\text{Sort}(FC_1) = \text{Sort}(FC_2) = \text{Sort}(FC)]$; **return** $Sync$, $C = \{c_1, c_2\} = \text{Sort}(FC)$
end function
procedure EMBED(S, m)
  $latest \leftarrow 0$; $index \leftarrow 2$; $S_w \leftarrow S$; $Risk \leftarrow$ {punct., stopwords, subwords}
  while $index < N - f$ **do**
    $ctx \leftarrow \{t_1..t_{index+1}\}$ in S_w
    if $t_{index} \in Risk$ **then**
      $index \leftarrow index + 1$; **continue**
    end if
    $Sync, C \leftarrow$ ST($index, ctx$); $Subst \leftarrow (t_{index} \in C) \wedge Sync$
    if $Subst$ **and** $index - latest \neq f + 1$ **then** # substitutability test
      for $c \in C$ **do**
        $new \leftarrow \{t_1..c, t_{index}\}$; $Sync', C' \leftarrow$ ST($index - 1, new$)
        if $t_{index-1} \in C' \wedge Sync'$ **then** $Subst \leftarrow$ False
      end for
    end if
    if $Subst$ **then**
      take next unembedded bit $signal$ of m; $t_{index} \leftarrow c_1$ if $signal = 0$ else c_2 # Eq. (7)
      $latest \leftarrow index$; $index \leftarrow index + f + 1$
    else
      $index \leftarrow index + 1$
    end if
  end while
  return S_w
end procedure
procedure EXTRACT(S_w)
  same loop; at each substitutable t_{index} output $signal = 0$ if $t_{index} = c_1$ else 1
end procedure

\+ context-aware candidates with high semantic relatedness; recoverable bit payload; works on any text

− synchronicity and substitutability tests exclude many words, so low capacity; extraction fails if the text is changed enough to alter candidate sets; rule-based pattern

Algorithm 15 DeepTextMark (Munyer & Zhong 2023)

Require: Word2Vec, Universal Sentence Encoder (USE), NLTK POS tagger, inflection engine, BERT, n nearest words

procedure INSERT(text)
  for each sentence s **do**
    candidates ← words that are not punctuation, stopwords or whitespace
    for each candidate: n nearest Word2Vec vectors → replacement words; keep those with the same grammatical number and POS tag
    proposals ← sentences with candidate words replaced (single, multiple or any-word substitution)
    best ← $\arg\max_p \cos\big(\mathrm{USE}(s), \mathrm{USE}(p)\big)$; output *best* # no human in the loop
  end for
end procedure
procedure DETECTSENTENCE(s)
  e ← BERT(s); h ← Transformer(e); pool; dropout; FC_{20}; dropout; FC_1 → watermark score
  train with binary cross-entropy; first BERT frozen, then unfrozen with a smaller Adam rate
end procedure
procedure DETECTTEXT(sentences)
  majority vote over sentence decisions; $P(\text{correct}) = \sum_{x>\lceil n/2 \rceil} \binom{n}{x} p^x (1-p)^{n-x} \to 1$ for $p > 0.85$
end procedure

\+ blind detection (no original text); automatic, no human intervention; classifier gets near-perfect on long documents by voting

− one bit per text (zero-bit), no payload; needs a trained BERT detector; synonym substitution can be reversed by paraphrase; per-sentence accuracy must be high

Algorithm 16 Multi-bit watermark with invariant features (Yoo et al. 2023)

Require: cover text X, message m, infill model θ (BERT), top-k_1, $k_2 \le k_1$, NER, YAKE, spaCy parser, NLI model

function $g_1(X)$ # Phase 1: mask positions S from invariant features
  Keyword: keywords ← NER entities ∪ YAKE terms; mask words adjacent to keywords
  Syntactic: parse dependencies; mask dependency types in a fixed order until the target number of masks is reached
  never mask keywords **return** S
end function
procedure DEPENDENCYORDER(held-out sentences) # Alg. 1 of the paper
  for each sentence X, each word x_i **do**
    $X' \leftarrow \mathrm{Infill}(X_{\setminus i})$; $s \leftarrow \mathrm{NLI}(X', X)$; $D[x.\mathrm{dep}].\mathrm{append}(s)$
  end for
  return dependency types sorted by mean entailment score, descending
end procedure
function $g_2(X, S)$ # Phase 2: valid watermark set
  for each mask $s_i \in S$: T_i ← top-k_1 infill tokens; remove punctuation, subwords, stopwords; keep top-k_2, sort alphabetically
  $\mathbf{T} \leftarrow T_{s_1} \times \cdots \times T_{s_j}$; $\mathbf{X}$ ← texts with tokens substituted
  $\mathbf{X}_{wm} \leftarrow \{X_i \in \mathbf{X} \mid g_1(X_{wm}) = g_1(X_i)\}$; assign a bit string to each element
end function
procedure EMBED(X, m)
  $S \leftarrow g_1(X)$; $\mathbf{X}_{wm} \leftarrow g_2(X, S)$; **return** the element whose assigned bits equal m
end procedure
procedure EXTRACT($\tilde{X}_{wm}$)
  $\tilde{S} \leftarrow g_1(\tilde{X}_{wm})$; $\tilde{\mathbf{X}}_{wm} \leftarrow g_2(\tilde{X}_{wm}, \tilde{S})$; **return** bits of $\tilde{X}_{wm}$ in that set
end procedure
procedure TRAIN ROBUST INFILL
  for corrupted $\tilde{X}$ (insert/delete/substitute round($CR \cdot N$) words, CR = 2.5–5%, similarity ≥ 0.98): p_i ← target over top-k_1 tokens on $X_{\setminus i}$ (renormalised, detached); $\tilde{p}_i \leftarrow$ prediction on $\tilde{X}_{\setminus i}$
  minimise reverse KL $\sum_i D_{KL}(\tilde{p}_i \,\|\, p_i)$ over masked positions chosen by g_1
end procedure

\+ multi-bit payload; mask positions depend on features that survive corruption ($R_{g_1} \approx$ 0.92–0.97 vs. 0.61–0.66 for ContextLS); error-free extraction without corruption; robust infill model improves robustness

− needs NER, parser, NLI and infill models; g_2 (infill output) is much less robust than g_1 unless the infill model is fine-tuned; synonym-level edits, small capacity per word

Algorithm 17 Classical character and lexical schemes (survey-level description only)

Require: payload bits; sources not on arXiv, so these steps follow Wu et al. Sec. 5.1.3, not the original papers

procedure EASYMARK(text, bits) # Rizzo et al. 2016
    replace regular spaces U+0020 with another blank code point (e.g. U+2004), or use variant selectors, homoglyphs or substrings of slightly different length, chosen by the bit to embed
end procedure
procedure UNISPACHI(text, bits) # Por et al. 2012
    insert chosen Unicode space characters between sentences, words, lines and paragraphs to carry the bits
end procedure
procedure T-LEX(text, payload) # Winstein 1998
    replace words with synonyms until the word-choice hash of the text encodes the payload
end procedure
procedure TOPKARA(text, payload) # Topkara et al. 2006
    WordNet synonym substitutions, then further changes to maximise ambiguity and stay near the distortion threshold
end procedure
procedure XIANG(text, payload) # Xiang et al. 2018
    quantise payload-carrying synonyms into binary sequences; compress with adaptive binary arithmetic coding; embed by substitution (reversible)
end procedure

+ character methods: invisible, work on short text and any format (Easymark); lexical methods: no model needed

− systematic patterns that are easy to find and strip (normalising Unicode, re-typing, paraphrase); synonym methods ignore sentence-level meaning

Sequence-to-sequence watermarks

Algorithm 18 AWT (Abdelnabi & Fritz 2021)

Require: sentence $S = \{W_0..W_n\}$, message $M = \{b_0..b_q\}$ ($q = 4$ bits per ≈80-word segment), encoder E, decoder D (transformers), message decoder $\mathcal{M}$, discriminator A, weights w_*

function HIDE(S, M)
    $e \leftarrow \text{AvgPool}_t(E(S)) + \text{Linear}(M)$ # shared embedding
    $S' \leftarrow \text{GumbelSoftmax}(D(\text{shifted } S,\ e))$ # one-hot fwd, soft bwd
end function
function REVEAL(S')
    return $\hat{M} = \text{Linear}(\text{AvgPool}_t(\mathcal{M}(S')))$
end function
procedure TRAIN STAGE 1
    $L_{rec} \leftarrow \text{CE}(S', S)$; $L_m \leftarrow \text{BCE}(\hat{M}, M)$
    $L_{disc} \leftarrow -\log A(S) - \log(1 - A(S'))$; $L_A \leftarrow -\log A(S')$
    minimise $L_1 = w_A L_A + w_{rec} L_{rec} + w_m L_m$ (A minimises L_{disc} alternately; $w_m \approx 5\times$ others)
end procedure
procedure FINE-TUNE STAGE 2
    $L_{sem} \leftarrow \|F(S) - F(S')\|$ # F: InferSent; $L_{LM} \leftarrow -\sum_i \log p_{LM}(W'_i \mid W'_{<i})$ # AWD-LSTM
    minimise $L_2 = w_A L_A + w_{rec} L_{rec} + w_m L_m + w_{sem} L_{sem} + w_{LM} L_{LM}$
end procedure
procedure VERIFY(document)
    split into segments; decode $\hat{M}$ per segment; $k \leftarrow$ matching bits against expected sequence out of n
    $p = \Pr(X \geq k \mid H_0) = \sum_{i \geq k} \binom{n}{i} 0.5^n$; reject H_0 if $p < 0.05$ (or 0.01)
end procedure

+ end-to-end learned; adversarial discriminator gives secrecy; preserves semantics

− only 4 bits per ≈80-word segment, so long text needed; needs training; weaker removal resistance than REMARK-LLM (per Wu et al.)

Algorithm 19 REMARK-LLM (Zhang et al. 2023)

Require: text $T = \{T_1..T_t\}$, binary signature M, Seq2Seq $(\mathbf{S}_e, \mathbf{S}_d)$, layers $\mathbf{R}_m, \mathbf{R}_n, \mathbf{R}_e$, extractor $\mathbf{E}$ (transformer), w_1, w_2, w_w, w_t

function ENCODE(T, M)
  $\mathcal{S}_e(T + M) \leftarrow \mathbf{S}_e(T)$ (norm. $\mathbf{R}_n$) $+ \mathbf{R}_n(\mathbf{R}_m(M))$ # message added to every token
  return $\mathcal{S}(T + M) = \mathbf{S}_d(\mathcal{S}_e(T + M))$ # distribution over V
end function
function REPARAM($\mathcal{S}_i$)
  return $\hat{\mathcal{S}}_i = \dfrac{\exp((\log \mathcal{S}_i + g_i)/\tau)}{\sum_{j=1}^{|V|} \exp((\log \mathcal{S}_j + g_j)/\tau)}$, $g_i \sim$ Gumbel$(0, 1)$
end function
function DECODE($\hat{\mathcal{S}}$)
  return $M' = \mathbf{E}(\mathcal{H})$, $\mathcal{H} = \mathbf{R}_e(\hat{\mathcal{S}})$
end function
procedure TRAIN
  mask input with $T_M \in \{0, 1\}^{|T|}$ each epoch
  $L_S \leftarrow -\frac{1}{|T|} \sum_{i=1}^{|T|} \sum_{j=1}^{|V|} T_{ij} \log \mathcal{S}_{ij}(T \cdot T_M + M)$
  $\hat{\mathcal{S}}_t \leftarrow$ randomly drop/add/replace tokens in $\hat{\mathcal{S}}$; $M'_t \leftarrow \mathbf{E}(\mathbf{R}_e(\hat{\mathcal{S}}_t))$
  $L_M \leftarrow w_w \sum_i |M_i - M'_i| + w_t \sum_i |M_i - M'_{t,i}|$; minimise $L = w_1 L_S + w_2 L_M$
end procedure
procedure INSERT($\bar{T}, \bar{M}, \boldsymbol{\tau}, B, K$)
  $best \leftarrow 0$; $\bar{T}_{wm} \leftarrow$ None
  for $k = 1..K$ **do**
    mask $\bar{T}_M$; $\mathcal{S} \leftarrow$ ENCODE($\bar{T} \cdot \bar{T}_M, \bar{M}$); $\mathcal{S}_{\text{noisy},i} \leftarrow \mathcal{S}_i +$ Gumbel$(\mathcal{S}_i, \tau_k)$
    $\{T_{ki}\} \leftarrow$ BEAMSEARCH($\mathcal{S}_{\text{noisy}}, B$)
    for each T_{ki} **do**
      $a \leftarrow$ Accuracy$(\mathbf{E}(T_{ki}), \bar{M})$; **if** $a > best$ **then** $best \leftarrow a$, $\bar{T}_{wm} \leftarrow T_{ki}$
    end for
  end for
  return $\bar{T}_{wm}$
end procedure
procedure EXTRACT($\bar{T}_{wm}$)
  $\bar{M}' \leftarrow \mathbf{E}(\mathbf{R}_e(\bar{T}_{wm}))$; $z \leftarrow \dfrac{|N| - \mu}{\sigma}$, $\mu = |M|p$, $\sigma^2 = |M|p(1-p)$, $p = 0.5$, $|N| =$ matched bits
end procedure

\+ more than twice the signature bits of AWT; trained against drop/add/replace noise; works on black-box LLM output

− heavy training; insertion needs K beam searches with the extractor in the loop; assumes the adversary lacks the private models

Not yet sourced: Aaronson's original EXP (blog; only the Kuditipudi variant above), SynthID-Text, and the original papers behind the survey-level box (Easymark, UniSpaChi, T-Lex, Topkara, Xiang; not on arXiv). The attack paper Lucas & Havens 2023 (backdoor trigger search) is not a watermarking scheme and is only cited in the pros/cons.

3 Statistics-based detection: count-based n-gram methods

Sources: Gallé et al. 2021 (arXiv 2111.02878) and Hamed & Wu 2023 (arXiv 2308.11767), both described in Wu et al. Sec. 5.2. Neither needs the generator or a key; the signal is that LLM text repeats high-order n-grams more (Gallé) or reuses a narrower vocabulary of technical bigrams (Hamed & Wu).

Algorithm 20 Repeat-based self-training detector (Gallé et al. 2021)

Require: document collection D, number of classifiers K, $minLen = 20$ chars, $minOcc = 3$

function DETECTION(D, K)
  $repeats \leftarrow$ SUPERMAXIMAL($D, minLen, minOcc$) # raw text, suffix array + LCP, linear time
  for $k = 1..K$ **do**
    $subset \leftarrow$ random 20 repeats
    $pos \leftarrow \{d \in D : \exists r \in subset,\ r \in d\}$ # pseudo-labelled machine
    $neg \leftarrow$ random $|pos|$ documents from D # noisy (unsupervised) or true human text (weakly supervised)
    $clf_k \leftarrow$ TRAIN(pos, neg) # binary classifier
  end for
  $score(d) \leftarrow \sum_{k=1}^{K} \mathbf{1}\left[clf_k(\text{machine} \mid d) > clf_k(\text{human} \mid d)\right]$ for all $d \in D$
  return D sorted by $score$ # ranking, not a per-document decision
end function

\+ unsupervised, no generator access, linear-time repeats

− ranks a *collection*, cannot judge a single text; a single repeat is weak evidence; needs many documents from the same generator; thresholds (20 chars, 3 occurrences) fixed ad hoc

A repeat r is *super-maximal* if it occurs $\geq minOcc$ times and no repeated string contains it (if $a{\cdot}r$ or $r{\cdot}a$ is also repeated for some symbol a, r is not super-maximal). It works on a *collection*, not on a single text.

Algorithm 21 xFakeBibs bigram-network detector (Hamed & Wu 2023)

Require: training set (100 real abstracts), 10 calibration folds of 100 real abstracts each, article a to classify

procedure TRAIN
  build bigram network $\mathcal{G}_{\text{train}}$ from TF-IDF bigrams (words = nodes, bigram = edge); $LCC \leftarrow$ largest connected component
end procedure
procedure CALIBRATE
  for each fold $f = 1..10$ **do**
    for each article $a \in f$ **do**
      $B \leftarrow$ TF-IDF bigrams of a; $c_a \leftarrow 0$
      for $b \in B$ **do**
        if b adds an edge between existing nodes of LCC (no new node) **then** $c_a \leftarrow c_a + 1$
      end for
      reset the training network to its original state
    end for
    $\bar{c}_f \leftarrow$ mean of c_a over the fold
  end for
  $lo \leftarrow \min_f \bar{c}_f$; $hi \leftarrow \max_f \bar{c}_f$ # reported: 21.96 and 25.12
end procedure
function CLASSIFY(a)
  $c_a \leftarrow$ as above **return** POSITIVE (real) if $lo \le c_a \le hi$ else NEGATIVE (fake)
end function

+ simple, interpretable; 98/100 ChatGPT abstracts flagged

− one domain (PubMed Alzheimer's), GPT-3.5, 100 articles; bounds calibrated on same domain; 2 missed; single-article detection remains hard; says little about edited or short text

Structural ratio used in the bulk analysis: Ratio = $\frac{\#\text{newly introduced edges (no new nodes)}}{\#\text{original edges}}$.

Caveat on the numbers. Wu et al. summarise the result as "ChatGPT accounted for only 23% of the bigram content". In the paper the *23%* is the share of new edges that join the training network's largest connected component without adding nodes (real folds: 51–70%); the plain bigram overlap with the training data is 9% for ChatGPT vs. 19–22% for the real folds. The 98/100 detection is on one domain (PubMed abstracts on Alzheimer's disease), 100 articles, GPT-3.5, with the bounds calibrated on the same domain, so it says little about domain shift or short or edited text.

4 Adversarial attacks on detectors

Sources: Wu et al. (CL 2025) Sec. 8.2 gives the taxonomy (paraphrase, adversarial, prompt, training-threat attacks); the exact procedures below come from the original papers in `articles/pdfs/` (dipper_paraphrase, sadasivan, deepwordbug, wolff_attacking, shi_redteam, lu_sco, escape, henrique_parrots, kulkarni_grover). In every box green + = what makes the attack work, red − = its limits. Common setting: detector $f(y) \in [0,1]$ (higher = more likely machine), threshold τ, attacker wants $f(y') < \tau$ while y' keeps the meaning and quality of y. Quality is measured by perplexity (e.g. OPT-13B) or paraphrase similarity (P-SP), detection by TPR at fixed low FPR.

Paraphrase attacks

Algorithm 22 DIPPER paraphrase attack (Krishna et al. 2023)

Require: paraphraser P = T5-XXL (11B), lexical diversity L and order diversity $O \in \{0, 20, \ldots, 100\}$, prompt x, machine text y

procedure BUILDTRAININGDATA(two translations p, q of one novel) # 6.3M pairs
  align sentences of p and q with Needleman–Wunsch, similarity = P-SP # dynamic programming
  choose a span $p_i..p_j$ and its aligned span $q_{i'}..q_{j'}$
  $q^{sh} \leftarrow$ random shuffle of the sentences $q_{i'}..q_{j'}$
  $L \leftarrow$ code from unigram-overlap F1 of $(p_i..p_j,\ q^{sh})$; $O \leftarrow$ code from Kendall-τ of overlapping words
  round L, O to $\{0, 20, \ldots, 100\}$ # L=20 ≈ 20% of words changed
  example: input = "lexical= L, order= O $\oplus$ $p_1..p_{i-1} \oplus$ q^{sh} $\oplus\ p_{j+1}..p_N$", target = $p_i..p_j$
  fine-tune P on all examples (max. 3 sentences per example)
end procedure
procedure ATTACK(x, y)
  split y into blocks of 3 sentences
  for each block b in order **do**
    input $\leftarrow$ "lexical= L, order= O x b " # paper text is cut off here; presumably earlier paraphrased blocks are kept as context
    $b' \leftarrow$ sample from P(input) with nucleus sampling, $p = 0.75$
  end for
  return concatenation of all b'
end procedure

+ discourse-level and context-aware, so meaning is kept (88–99% of paraphrases score above the median human paraphrase in P-SP); two knobs let the attacker change only as much as needed; DetectGPT accuracy 70.3%→4.6% at 1% FPR, KGW watermark 100%→57.2% (L=60, O=60); the paraphraser must differ from the watermarked model, otherwise the paraphrase is watermarked too

− needs an 11B model; a single pass leaves the watermark only partly removed (57% still detected); best results need several samples and a queryable detector; defeated by retrieval over the API's past outputs

Algorithm 23 Recursive paraphrasing (Sadasivan et al. 2023)

Require: machine text $S = (s_1..s_n)$, optional context C, rounds R, paraphraser f (DIPPER or LLaMA-2-7B-Chat: whole passage with context; Parrot T5: sentence by sentence), optional black-box detector D

procedure RECURSIVE(S)
  $S_0 \leftarrow S$
  for $r = 1, \ldots, R$ **do**
    $S_r \leftarrow f_{strong}(S_{r-1}, C)$ **or** $S_r \leftarrow (f_{weak}(s_1), \ldots, f_{weak}(s_n))$ # S_r = "ppr"
  end for
  return S_R
end procedure
procedure BESTOFPP(S) # needs black-box query access to D
  return $\arg\min_{r \in \{1..R\}} D(S_r)$ # paraphrase with the worst detector score
end procedure

+ each round keeps the previous one as input, so the watermark/style signal is removed gradually; OPT-13B watermark TPR@1%FPR 99.8%→80.7% (DIPPER) and 54.6% (LLaMA-2); best-of-pp on OPT-1.3B 99.8%→4.0%; also breaks the retrieval detector of Krishna et al.

− perplexity rises (5.5 → 8.7 / 10.5) and meaning drifts with rounds; short texts (50–100 tokens) are the easy case for the attacker, long texts keep more signal; best-of needs the detector

Algorithm 24 Retrieval defense against paraphrasing (Krishna et al. 2023)

Require: LLM API f_{LM}, encoder f_{ret} (P-SP, or BM25 with unigram-F1 normalisation), threshold T

procedure LOG(x_i) # API side, every request
  $y_i \leftarrow f_{LM}(x_i)$; store $\mathbf{y}_i \leftarrow f_{ret}(y_i)$ in database $\mathbf{Y}$ # up to 15M entries tested
end procedure
procedure DETECT(y')
  $\mathbf{y}' \leftarrow f_{ret}(y')$; $score \leftarrow \max_i \text{sim}(\mathbf{y}', \mathbf{y}_i)$ # exact copy gives 1.0
  return $score \geq T$
end procedure

+ catches 80–97% of paraphrased outputs at 1% FPR; independent of text quality, so the impossibility argument does not apply

− needs the provider to store every output; privacy cost; broken by recursive paraphrasing (Sadasivan et al.) and spoofed 100% of the time even with a private database

Spoofing (the opposite goal: make human text look machine-made)

Algorithm 25 Watermark spoofing by learning a proxy green list (Sadasivan et al. 2023)

Require: query access to watermarked LM M_{wm} (KGW), word set W ($|W| = 181$ most common words), $N = 10^6$ queries

procedure LEARNPROXY
  $count[w][v] \leftarrow 0$ for all $w \in W$, $v \in W$
  for each of N generations g from M_{wm} **do**
    for each adjacent pair (w, v) in g with $w, v \in W$ **do**
      $count[w][v] \leftarrow count[w][v] + 1$
    end for
  end for
  $\tilde{G}_w \leftarrow$ tokens v with high $count[w][v]$ # rule not given in the text I read; see paper
end procedure
procedure SPOOF(human author)
  write text whose token v after w is in $\tilde{G}_w$ as often as possible
end procedure

+ one-off learning, then reusable; detector AUROC on spoofed text 99.8%→1.3% (human text falsely flagged)

− only covers a small vocabulary subset; needs 10^6 queries to the target; the same idea also gives a scrubbing recipe (avoid $\tilde{G}_w$)

Adversarial perturbations of characters, words and embeddings

Algorithm 26 DeepWordBug (Gao et al. 2018; used on text detectors by Stiff & Johansson 2022)

Require: text $x = x_1 x_2 .. x_n$ (words), black-box classifier score F, scoring function S, character transform T, edit budget ϵ, λ

function SCORE($x_i; x$) # importance of word i for the class decision
  Replace-1: $F(x_1..x_i..x_n) - F(x_1..x'_i..x_n)$ # x'_i = replacement for x_i
  Temporal Head: $F(x_1..x_i) - F(x_1..x_{i-1})$
  Temporal Tail: $F(x_i..x_n) - F(x_{i+1}..x_n)$
  Combined: $\mathrm{THS}(x_i) + \lambda\,\mathrm{TTS}(x_i)$
end function
function TRANSFORM(word w) # returns the edit distance used
  pick one: **swap** two adjacent letters (cost 2) **or substitute** a letter by a random one **or delete** a random letter **or insert** a random letter (cost 1 each)
end function
procedure DEEPWORDBUG(x)
  $scores[i] \leftarrow S(x_i; x)$ for $i = 1..n$; sort indices by descending score: $L_1..L_n$
  $x' \leftarrow x$; $cost \leftarrow 0$; $j \leftarrow 1$
  while $cost < \epsilon$ **do**
    $cost \leftarrow cost + \mathrm{TRANSFORM}(x'_{L_j})$; $j \leftarrow j + 1$ # edits the word in place
  end while
  return x'
end procedure

\+ black-box, at most $O(n)$ queries; edits are tiny so a human still reads the text; halved the OpenAI RoBERTa detector's performance (Stiff & Johansson)
− misspellings are easy to spot and to normalise with a spell-check; ordering signals (watermark green/red) are only hit if the edited word is scored

Algorithm 27 Homoglyph and misspelling attacks (Wolff 2020)

Require: text y, homoglyph map H (e.g. Latin a → Cyrillic U+0430, e → U+0435), rate $\rho_h = 1.5\%$ of characters, list M of common human misspellings (Wikipedia), rate $\rho_m = 5\%$ of words

procedure HOMOGLYPH(y)
  choose $\rho_h|y|$ characters of the source letters at random; replace each by its look-alike from H
  return y' # looks identical, but the tokenizer sees different tokens
end procedure
procedure MISSPELL(y)
  choose $\rho_m \cdot$ #words words that appear in M at random; replace each by a listed misspelling
  return y'
end procedure

\+ no model needed, black-box, invisible (homoglyph) or human-like (misspelling); RoBERTa detector recall 97.44%→13.57% with a/e Cyrillic at 1.5% rate; replacing all occurrences: 0.26% (homoglyph) and 22.68% (misspelling), numbers as given in the survey
− normalising Unicode (NFKC does not map Cyrillic to Latin, a confusables table does) or spell-checking removes it; GLTR does not accept many Unicode characters, so only a restricted homoglyph set worked there

Algorithm 28 LLM word substitution against detectors (Shi et al. 2023, red teaming)

Require: output $Y = [y_1..y_m]$, detector f (classifier, KGW, or DetectGPT), helper LLM G', budget ϵ (at most ϵm words changed), n candidates per word

function CANDIDATES(y_k, Y) # set of $\leq n$ context-fitting words, may be empty
  return words from prompting G': "give n substitution words that do not change the meaning of the sentence" (ChatGPT); for LLaMA a completion "Synonyms of the word y_k in the above sentence are:"
end function
procedure QUERYBASED(Y) # needs black-box scores of f; solves $\min_{Y'} f(Y')$
  population $\leftarrow$ copies of Y with random words replaced by CANDIDATES
  for each generation **do**
    keep the elites (lowest f); build the next population by crossover of the elites
  end for
  return best element # evolutionary search of Alzantot et al. 2018; population size and number of generations not read
end procedure
procedure QUERYFREE(Y)
  if f is DetectGPT or a classifier **then**
    replace ϵm random words using CANDIDATES
  else # KGW: target green tokens
    estimate the prediction entropy of every token with an LM M (prefix or masked text)
    replace the ϵm highest-entropy tokens (they are the most likely to be green) using CANDIDATES
  end if
end procedure

\+ LLM candidates fit the context, so no extra synonym filter; the KGW variant needs no detector query; text stays meaningful

− needs repeated queries to a helper LLM per text; query-based variant needs detector scores; effect bounded by the budget ϵ

Algorithm 29 Frankenstein, synonym and FGSM-embedding perturbations of Grover (Kulkarni et al.)

Require: base article a (keep its metadata: title, date, authors, URL strongly affect Grover), source articles, detector gradient access (FGSM part)

procedure FRANKENSTEIN(a, source, fraction q)
  split both into sentences (NLTK); choose random sentence indices in a and in source
  substitute (or insert) the chosen source sentences into a until a fraction q is replaced
end procedure
procedure SYNONYMSWAP(a)
  repeat n times: replace one random word by a WordNet synonym, producing articles with $1, 2, \ldots, n$ swaps
end procedure
procedure FGSMTOKENPAIRS(article, steps $\epsilon \in \{0.001, 0.1, 1, 5\}$)
  $E \leftarrow$ embedding table; $G \leftarrow \partial\,\text{loss}/\partial E$ for the article; $E^{adv} \leftarrow E + \epsilon \cdot G$
  for each token t used in the article **do**
    $v \leftarrow \arg\max_u \cos(E_u, E_t^{adv})$ over normalised embeddings
    if $\cos > 0.18$ **then**
      add pair $(t \rightarrow v)$ to the replacement dictionary
    end if
  end for
  re-encode the article and replace the tokens in the dictionary
end procedure

\+ FGSM variant changed one or two tokens and fooled Grover in both directions (human→machine and machine→human); the substituted tokens are the same, so perplexity stays about the same

− requires gradient access to the detector (white-box); synonym swapping had no effect on Grover; a course report, not a peer-reviewed result

Prompt attacks

Algorithm 30 SICO: substitution-based in-context example optimisation (Lu et al. 2024)

Require: LLM, proxy detector giving $P_{AI}(\cdot)$, triplets $D = \{(x_{ic}^k, y_{AI}^k, y_{human}^k)\}_{k=1}^K$, evaluation inputs X_{eval}, iterations N, task instruction p_{task}, paraphrase instruction p_{para}

function UTILITY(prompt p)
  return $1 - \frac{1}{|X_{eval}|} \sum_{x \in X_{eval}} P_{AI}\big(\text{LLM}(p \oplus x)\big)$
end function
function GREEDYOPT(text y, substitutions C) # $C_{i,j}$ = j-th word or sentence substitute for unit y_i
  for each unit y_i: $C_{i,*} \leftarrow \arg\min_j P_{AI}\big(\text{SUB}(y_i, C_{i,j})\big)$ # best single substitution
  apply all best substitutions $C_{i,*}$ to y one after another; **return** y
end function
procedure SICO
  $t_{feature} \leftarrow$ LLM summary of how human text differs from AI text, from the pairs (y_{AI}^k, y_{human}^k)
  $y_{ic}^k \leftarrow \text{LLM}(t_{feature} \oplus p_{para} \oplus y_{AI}^k)$ for all k # start from human-style paraphrases
  $p^* \leftarrow t_{feature} \oplus p_{task} \oplus \{(x_{ic}^k, y_{ic}^k)\}$
  for $n = 1..N$ **do**
    for $k = 1..K$ **do**
      $C^k \leftarrow$ sentence-level (LLM paraphrases using $t_{feature}$) or word-level (WordNet synonyms, same POS, filtered by a masked LM) substitutions of y_{ic}^k, switching with n
      $\hat{y}_{ic}^k \leftarrow \text{GREEDYOPT}(y_{ic}^k, C^k)$
    end for
    $\hat{p} \leftarrow t_{feature} \oplus p_{task} \oplus \{(x_{ic}^k, \hat{y}_{ic}^k)\}$
    if $\text{UTILITY}(\hat{p}) > \text{UTILITY}(p^*)$ **then**
      $y_{ic}^k \leftarrow \hat{y}_{ic}^k$; $p^* \leftarrow \hat{p}$
    end if
  end for
  return p^* # reusable prompt template; at test time put any task input x after it
end procedure

+ no fine-tuning, works on a black-box LLM; average AUC drop of 0.54 and better text quality than paraphrase attacks (survey summary of the paper); *SICO-Para* reuses it as a paraphraser

− needs a proxy detector and many LLM calls during the one-off optimisation; transfer to other detectors is imperfect; training detectors on SICO outputs restores much of the performance (Sec. 6.1 of the paper)

The survey spells the name "Substitution-based Contextual Example Optimization"; the paper's title uses *in-context* and the abbreviation is SICO.

Algorithm 31 Instructional-prompt attack (Shi et al. 2023, red teaming)

Require: generator G (ChatGPT), black-box detector f, n training prompts $X_1..X_n$, top-K K, iterations T

procedure SEARCHPROMPT
  $X_{ins} \leftarrow$ "Meanwhile please imitate the writing style and wording of the following passage:"; $X_{ref} \leftarrow \emptyset$
  queue $O \leftarrow$ the n outputs of G without suffix, sorted by detector score f (lowest first)
  for $t = 1..T$ **do**
    update X_{ref}**:** pop the K best texts from O; candidates $X_p = [X_{ins}, \text{text}]$
    update X_{ins}**:** ask G for K rewrites of X_{ins}; candidates $X_p = [\text{rewrite}, X_{ref}]$
    keep the candidate minimising $\frac{1}{n}\sum_i f\big(G([X_i, X_p])\big)$; push all its outputs into O
    discard candidates for which more than 20% of the (non-stop) words of X_{ref} appear in the output # copy guard
  end for
  return $X_p = [X_{ins}, X_{ref}]$
end procedure
procedure ATTACK(X)
  return $G([X, X_p])$ # test time: no detector queries
end procedure

+ query-free at test time; one prompt works for a whole dataset; no change to the generated text itself, only to the input

− search needs many detector queries; only tested on instruction-following LLMs against classifier-based detectors; style is imitated, so content is not guaranteed to match the original

Training threat models

Algorithm 32 EScaPe: evasive soft prompts (Kumarage et al. 2023)

Require: frozen source LM PLM^s (policy π), proxy detector d_ϕ, writing prompts X, k trainable virtual tokens $P^s = (p_1..p_k)$, $p_i \in \mathbb{R}^{e_s}$, penalty β

procedure LearnSoftPrompt
  for each PPO step **do**
    $Y \leftarrow PLM^s([P^s;\ \mathrm{emb}(X)])$ # virtual tokens prepended to the prompt embeddings
    $\hat{z} \leftarrow d_\phi(Y)[\mathrm{Human}]$ # probability that the detector says human
    $R \leftarrow \hat{z} - \beta\,\mathrm{KL}(\pi \,\|\, PLM^s)$ # KL keeps the text coherent and on topic
    update only P^s by the PPO policy gradient with reward R
  end for
end procedure
procedure Transfer(target LM PLM^t)
  $P^t \leftarrow g_\theta(P^s) = W_2\,\sigma(W_1 P^s + b_1) + b_2$ # 2-layer projector $\mathbb{R}^{e_s} \to \mathbb{R}^{e_t}$, trained with the same RL loop (my reading of Eq. 7)
end procedure

\+ only k embeddings are trained, the LM stays frozen; transfers across LMs and across detectors; DetectGPT and OpenAI detector AUROC down by up to 40% (survey summary)

− needs a proxy detector and RL training per source model; soft prompts need access to the LM's embeddings (open weights); the quality of the evasive text is lower than the original

Algorithm 33 Fine-tuning against a detector, "hide and seek" (Da Silva Gameiro Henrique et al. 2023)

Require: generator G = GPT-2 small (117M), discriminator D = BERT base (110M), human reference corpus H (the data D was trained on), prompts = first 5 tokens of corpus entries

procedure HideAndSeek
  repeat
    $y \leftarrow G$(prompt) for a batch of prompts from H
    train D to separate H (human) from y (machine)
    update G by reinforcement from the critic D # AdamW instead of Adam; best of five restarts; exact reward form not read
  until converged
end procedure

\+ shows that if the attacker has the human data the detector was trained on, fine-tuning on that data makes the detector fail *even though all its labels are correct*

− small models and small datasets (MS COCO captions, EMNLP news); DP-GAN with RNNs collapses and was ruled out; the result assumes access to D's reference data

Survey-level attacks (original papers not read)

The survey lists these without algorithms; I could not fetch the originals, so treat the lines as pointers, not as exact procedures.

Algorithm 34 Attacks only described in Wu et al. Sec. 8.2

procedure Cutoff (Shen et al. 2020)
  crop part of the input or of its feature vector
end procedure
procedure Shuffle (Lee et al. 2020)
  randomly reorder the words of the input
end procedure
procedure Mutation (Liang et al. 2023)
  change characters or words
end procedure
procedure WordSwap (Shi & Huang 2020; Ren et al. 2019 PWWS; Crothers et al. 2022)
  replace words by contextually fitting ones, ranked by saliency × probability change # PWWS formula from memory, not verified
end procedure
procedure TextAttack (Morris et al. 2020)
  attack = objective function + constraints + transformation + search method
end procedure
procedure Syntax (Bhat & Parthasarathy 2020)
  split long sentences, drop definite articles, expand contractions ("that's" → "that is"), reformat paragraphs
end procedure
procedure WeightedSaliency (He et al. 2023b, MGTBench)
  probability-weighted word saliency to build adversarial examples that keep the meaning
end procedure
procedure RLEvasion (Nicks et al. 2023; Schneider et al. 2023)
  fine-tune Llama-2-7B / a generator with detector "humanity" scores as reward; OpenAI RoBERTa AUROC 0.84→0.62 (Nicks), BERT-based classifier down to 0.15 (Schneider)
end procedure
procedure PromptComplexity (Guo et al. 2023; Liu et al. 2023e)
  complex or varied prompts (few-shot, CoT) lower the detector's recall
end procedure
procedure AdvTrainingDefense (Antoun et al. 2023)
  train the detector on adversarial samples; this improves its recognition of the attack
end procedure

What this means for the thesis (my synthesis, not from the sources). Paraphrase and word-substitution attacks act on token identity and order, so they hit token-level watermarks (KGW, Unigram) first; sentence-level watermarks (SemStamp) and EXP-EDIT alignment should lose less; homoglyph and character attacks change the tokenisation, so they hit both watermark detection and perplexity-

based fingerprints but are removed by Unicode normalisation; prompt and training attacks shift the style and so target stylometric and likelihood-based fingerprints rather than the key-based watermark. The experiments should run each attack at several strengths (paraphrase rounds R, edit budget ϵ, diversity codes L, O) and report TPR at 1% and 0.1% FPR per signal and fused.

5 Article summaries

6 Ideas and future research